

Nama: Cornelius Nathanael Bedu

NIM: 2902594033

Penjelasan Tujuan Program

Tujuan program ini adalah untuk membuat sebuah aplikasi kamus digital bagi perusahaan "Boogle" yang berfungsi untuk mendokumentasikan dan melihat daftar kata gaul (*slang words*) baru dari internet.

Aplikasi ini menggunakan struktur data **Trie** untuk menyimpan datanya, dan memungkinkan penggunaanya untuk:

- Merilis (menyimpan) kata *slang* baru beserta maknanya, atau memperbarui maknanya jika kata tersebut sudah ada.
- Mencari makna dari sebuah kata *slang* secara spesifik.
- Melihat daftar kata *slang* yang memiliki awalan huruf (prefix) tertentu.

Menampilkan seluruh kata *slang* yang telah direkam secara berurutan sesuai abjad (alfabetis).

Penjelasan Logika Kode Program

1. Inisialisasi Library dan Konstanta

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <stdbool.h>
#include <ctype.h>

#define MAX_CHARS 256
#define MAX_SLANG_LEN 100
#define MAX_MEANING_LEN 500
```

Kode di atas saya pakai untuk memanggil *library* standar bawaan bahasa C/C++ yang dibutuhkan program. Misalnya <stdio.h> buat input-output dasar, <stdlib.h> untuk manajemen memori dinamis, dan <string.h> untuk mengatur teks. Saya juga membuat beberapa konstanta pakai sintaks #define. MAX_CHARS saya set 256 supaya struktur Trie-nya bisa menampung semua variasi karakter ASCII. Terus, MAX_SLANG_LEN dan MAX_MEANING_LEN saya pakai untuk membatasi panjang input kata dan deskripsinya agar pemakaian memori program tetap aman.

2. Pembuatan Struktur Data Trie

```
typedef struct SlangNode {
    struct SlangNode *charPaths[MAX_CHARS];
    char *wordDefinition;
```

```

        bool isCompleteWord;
    } SlangNode;

    int dictionarySize = 0;

```

Di sini saya membuat struktur data untuk Trie-nya dan saya beri nama SlangNode. Tiap *node* punya *array* charPaths sebanyak 256 yang fungsinya jadi cabang penunjuk ke huruf selanjutnya. Terus ada pointer wordDefinition buat menyimpan teks arti dari kata slang-nya, dan tipe data boolean isCompleteWord buat menandai apakah di posisi huruf tersebut sudah membentuk satu kata yang utuh atau belum. Saya juga menambahkan variabel global dictionarySize yang mulai dari 0, gunanya untuk menghitung otomatis ada berapa total kata yang sudah berhasil masuk ke kamus.

3. Fungsi Pembuat Node Baru

```

SlangNode* initSlangNode() {
    SlangNode* newNode = (SlangNode*)malloc(sizeof(SlangNode));
    newNode->isCompleteWord = false;
    newNode->wordDefinition = NULL;
    for (int i = 0; i < MAX_CHARS; i++) {
        newNode->charPaths[i] = NULL;
    }
    return newNode;
}

```

Fungsi ini tugasnya untuk membuat *node* huruf baru setiap kali program mau menyimpan data ke dalam Trie. Awalnya saya meminta alokasi memori kosong ke sistem menggunakan fungsi malloc sesuai ukuran SlangNode. Kemudian, status awalan kata utuhnya (isCompleteWord) saya set false dan teks deskripsinya dikosongkan (NULL). Supaya aman dan tidak ada *error* memori bocor, saya pakai *looping* for untuk memastikan ke-256 cabang hurufnya benar-benar putus atau NULL. Terakhir, fungsi ini bakal me-*return node* yang sudah bersih tersebut biar bisa dipakai.

4. Fungsi Bantuan Pembersih Input

```

void flushInput() {
    int keyPress;
    while ((keyPress = getchar()) != '\n' && keyPress != EOF);
}

void removeTrailingEnter(char *inputText) {
    int length = strlen(inputText);
    if (length > 0 && inputText[length - 1] == '\n') {
        inputText[length - 1] = '\0';
    }
}

```

Dua fungsi tambahan ini saya bikin untuk mengakali kelemahan bawaan saat kita mengambil

inputan di C/C++. Fungsi `flushInput()` menggunakan perulangan `getchar()` untuk membuang sisa karakter atau tombol *Enter* (`\n`) yang nyangkut di memori sistem. Ini penting agar pas *user* milih menu, inputannya nggak otomatis *ke-skip*. Kalau fungsi `removeTrailingEnter()`, saya pakai buat menghapus karakter *Enter* yang biasanya selalu ikut tersimpan pas kita pakai fungsi baca teks `fgets`. Logikanya simpel, saya cek huruf paling ujung belakang, kalau itu sebuah *Enter*, langsung saya timpa jadi `\0` biar batas akhir teksnya rapi.

5. Fungsi Validasi Kata dan Deskripsi (Tanpa Regex)

```
bool checkSlangFormat(char* term) {
    int length = strlen(term);
    if (length <= 1) return false;
    for (int i = 0; i < length; i++) {
        if (isspace(term[i])) return false;
    }
    return true;
}

bool checkMeaningFormat(char* meaningText) {
    int wordTally = 0;
    bool isReadingWord = false;
    for (int i = 0; meaningText[i] != '\0'; i++) {
        if (!isspace(meaningText[i])) {
            if (!isReadingWord) {
                wordTally++;
                isReadingWord = true;
            }
        } else {
            isReadingWord = false;
        }
    }
    return wordTally >= 2;
}
```

Karena di soal dilarang pakai *Regex*, saya membuat validasi manual sendiri. Fungsi pertama bertugas mengecek kata slang. Syaratnya harus lebih dari 1 huruf dan nggak boleh ada spasi. Jadi, program *nge-looping* ngecek tiap hurufnya satu per satu. Kalau di tengah jalan ketemu spasi (pakai fungsi `isspace`), fungsinya langsung *me-return false*. Buat fungsi kedua, itu untuk memastikan kalimat deskripsinya minimal ada 2 kata. Caranya program membaca huruf demi huruf. Kalau habis membaca spasi terus dia ketemu abjad lagi, itu dihitung sebagai awalan sebuah kata baru. Kalau jumlah kata yang terhitung ternyata 2 atau lebih, berarti formatnya sudah pas dan fungsi merespons *true*.

6. Fungsi Menyimpan dan Memperbarui Data (Insert/Update)

```
void saveSlangData(SlangNode* baseRoot, char* term, char* meaningText)
{
    SlangNode* tracker = baseRoot;
```

```

for (int i = 0; term[i] != '\0'; i++) {
    unsigned char letterIndex = term[i];
    if (tracker->charPaths[letterIndex] == NULL) {
        tracker->charPaths[letterIndex] = initSlangNode();
    }
    tracker = tracker->charPaths[letterIndex];
}

if (tracker->isCompleteWord) {
    free(tracker->wordDefinition);
    tracker->wordDefinition = strdup(meaningText);
    printf("\nSuccessfully updated a slang word.\n");
} else {
    tracker->isCompleteWord = true;
    tracker->wordDefinition = strdup(meaningText);
    dictionarySize++;
    printf("\nSuccessfully released new slang word.\n");
}
}

```

Fungsi ini dipakai untuk menyimpan kata slang dan deskripsinya ke dalam Trie. Saya mulai jalan dari *root* yang paling awal, terus memecah kata inputan jadi huruf-huruf. Tiap huruf saya cek cabangnya, kalau belum ada, saya buat *node* jalurnya. Pas perulangannya sudah mentok di huruf yang terakhir, saya mengecek nilai *isCompleteWord*-nya. Kalau nilainya *true* (yang berarti kata itu sudah pernah diinput *user* sebelumnya), memori deskripsi yang lama bakal dihapus pakai *free()* terus diganti sama duplikat memori deskripsi yang baru. Tapi kalau *false*, berarti itu benar-benar kata baru. Saya set statusnya jadi utuh, simpan deskripsinya, lalu tambahkan angka global *dictionarySize*.

7. Fungsi Pencarian Kata

```

SlangNode* findSlangNode(SlangNode* baseRoot, char* term) {
    SlangNode* tracker = baseRoot;
    for (int i = 0; term[i] != '\0'; i++) {
        unsigned char letterIndex = term[i];
        if (tracker->charPaths[letterIndex] == NULL) {
            return NULL;
        }
        tracker = tracker->charPaths[letterIndex];
    }
    return tracker->isCompleteWord ? tracker : NULL;
}

```

Fungsi ini saya buat untuk mencari titik lokasi sebuah kata di dalam tumpukan Trie. Cara kerjanya persis seperti fungsi *saveSlangData* di atas. Saya turun menyusuri cabang demi cabang menyesuaikan huruf inputannya. Bedanya, kalau di tengah jalan cabangnya tiba-tiba tidak ada (*NULL*), fungsi ini bakal langsung *me-return NULL* yang artinya kata itu gagal ditemukan. Tapi kalau pencariannya berhasil sampai ujung huruf terakhir, saya menggunakan

ternary operator (simbol tanda tanya) untuk mengecek secara pasti apakah di titik huruf itu status penanda katanya benar-benar menyala utuh. Kalau iya, data lokasinya dikembalikan agar deskripsinya bisa dipanggil.

8. Fungsi Cetak Urut Abjad (DFS Rekursif)

```
void displayAlphabetically(SlangNode* currentNode, char* wordTemp, int
level, int* orderNum) {
    if (!currentNode) return;

    if (currentNode->isCompleteWord) {
        wordTemp[level] = '\0';
        printf("%d. %s\n", (*orderNum)++, wordTemp);
    }

    for (int i = 0; i < MAX_CHARS; i++) {
        if (currentNode->charPaths[i] != NULL) {
            wordTemp[level] = i;
            displayAlphabetically(currentNode->charPaths[i], wordTemp,
level + 1, orderNum);
        }
    }
}
```

Supaya datanya bisa ditampilkan secara urut abjad atau berurutan leksikografis, saya membuat metode *Depth-First Search* (DFS) yang sifatnya rekursif. Triknya ada di perulangan for dari indeks 0 sampai 255. Karena dari sananya komputer memang menyusun nilai karakter secara urut abjad membesar, fungsi ini otomatis bakal menelusuri rute huruf yang paling awal (terkecil) duluan dengan memanggil fungsi dirinya sendiri. Tiap kali fungsi ini menemukan *node* yang kebetulan membentuk kata slang (*isCompleteWord*), semua abjad yang sudah ditampung sejauh itu bakal ditutup pakai \0 terus diprint ke layar berserta penomoran urutnya.

9. Fungsi Layar Tunggu & Awal Main()

```
void waitForEnter() {
    printf("Press enter to continue...\n");
    flushInput();
}

int main() {
    SlangNode* mainRoot = initSlangNode();
    int userMenu;
    char slangTerm[MAX_SLANG_LEN];
    char slangMeaning[MAX_MEANING_LEN];
    char printBuffer[MAX_SLANG_LEN];

    do {
        printf("\n=== Boogle Slang Dictionary ===\n");
```

```

printf("1. Release a new slang word\n");
printf("2. Search a slang word\n");
printf("3. View all slang words starting with a certain prefix
word\n");
printf("4. View all slang words\n");
printf("5. Exit\n");
printf("Choose menu: ");

if (scanf("%d", &userMenu) != 1) {
    flushInput();
    continue;
}
flushInput();

```

Fungsi `waitForEnter()` saya buat sekadar untuk menahan layar sebelum program kembali ke menu utama. Sedangkan bagian `main()` ini adalah titik awal kendali program. Di awal, saya inisialisasi *root* dasar untuk Trie-nya, lalu menyiapkan beberapa variabel penyimpanan teks. Agar antarmukanya muncul berulang kali, tampilannya saya masukkan ke dalam perulangan `do-while`. Saya menggunakan `scanf` untuk mengambil opsi angka menu. Di sini saya tambahkan sedikit validasi keamanan; kalau *user* iseng memasukkan huruf atau karakter asing, program bakal *nge-flush* sisa inputan memori yang nyangkut dan *me-restart* layar dengan perintah `continue`.

10. Menu 1: Merilis Kata Slang Baru

```

switch(userMenu) {
    case 1: {
        do {
            printf("Input a new slang word [Must be more than
1 characters and contains no space]: ");
            fgets(slangTerm, MAX_SLANG_LEN, stdin);
            removeTrailingEnter(slangTerm);
        } while (!checkSlangFormat(slangTerm));

        do {
            printf("Input a new slang word description [Must
be more than 1 word]: ");
            fgets(slangMeaning, MAX_MEANING_LEN, stdin);
            removeTrailingEnter(slangMeaning);
        } while (!checkMeaningFormat(slangMeaning));

        saveSlangData(mainRoot, slangTerm, slangMeaning);
        waitForEnter();
        break;
    }
}

```

Masuk ke `switch case`, menu pertama ini gunanya untuk memasukkan kata slang baru. Saya membungkus instruksi inputnya pakai `do-while` supaya program memaksa *user* mengulang

masukannya kalau format katanya belum valid dan belum lolos dari kedua fungsi *checking* yang dibuat sebelumnya. Setelah inputan dinyatakan benar, teks kata dan deskripsinya dikirim ke fungsi `saveSlangData` buat disimpan ke dalam Trie. Oh iya, *case 1* ini saya kurung pakai tanda kurung kurawal { }. Ini perlu dilakukan di C++ supaya memori variabel baru di dalamnya punya batas (*scope*) sendiri dan nggak bikin program *error* saat di-kompilasi.

11. Menu 2: Mencari Kata Slang

```
case 2: {
    do {
        printf("Input a slang word to be searched [Must be
more than 1 characters and contains no space]: ");
        fgets(slangTerm, MAX_SLANG_LEN, stdin);
        removeTrailingEnter(slangTerm);
    } while (!checkSlangFormat(slangTerm));

    SlangNode* searchResult = findSlangNode(mainRoot,
slangTerm);
    if (searchResult) {
        printf("\nSlang word   : %s\n", slangTerm);
        printf("Description : %s\n\n",
searchResult->wordDefinition);
    } else {
        printf("\nThere is no word \"%s\" in the
dictionary.\n\n", slangTerm);
    }
    waitForEnter();
    break;
}
```

Menu kedua berfungsi untuk mencari kata slang. Sama dengan fitur pertama, inputan kata dari *user* akan divalidasi dulu pakai perulangan sampai formatnya benar tanpa spasi. Setelah valid, kata tersebut dikirim ke fungsi pencarian. Laporan hasil fungsinya ditampung di pointer `searchResult`. Kalau pointer itu ternyata ada wujudnya (nggak *NULL*), program bakal mengambil dan mem-print isi deskripsinya ke layar. Tapi kalau hasil pointernya *NULL*, program langsung mencetak notifikasi kalau kata tersebut nggak terdaftar di kamus. Kemudian layar di-*pause* sebentar nunggu *user* tekan *Enter*.

12. Menu 3: Pencarian Berdasarkan Awalan (*Prefix*)

```
case 3: {
    printf("Input a prefix to be searched: ");
    fgets(slangTerm, MAX_SLANG_LEN, stdin);
    removeTrailingEnter(slangTerm);

    SlangNode* prefixTracker = mainRoot;
    bool isPrefixFound = true;
    for (int i = 0; slangTerm[i] != '\0'; i++) {
```

```

        unsigned char letterIndex = slangTerm[i];
        if (prefixTracker->charPaths[letterIndex] == NULL)
        {
            isPrefixFound = false;
            break;
        }
        prefixTracker =
prefixTracker->charPaths[letterIndex];
    }

    if (!isPrefixFound) {
        printf("\nThere is no prefix \"%s\" in the
dictionary.\n\n", slangTerm);
    } else {
        printf("\nWords starts with \"%s\":\n",
slangTerm);

        int currentNum = 1;
        strcpy(printBuffer, slangTerm);
        displayAlphabetically(prefixTracker, printBuffer,
strlen(slangTerm), &currentNum);
        printf("\n");
    }
    waitForEnter();
    break;
}

```

Pada menu ketiga ini kita diminta nge-print semua kata slang dengan awalan (*prefix*) tertentu. Logikanya saya buat efisien; program nggak menelusuri dari nol sambil mengecek satu per satu kata yang ada di dalam kamus. Melainkan, saya membuat perulangan for singkat untuk berjalan mengikuti awalan huruf yang *user* masukan saja. Kalau di pertengahan rute huruf ternyata jalurnya buntu (NULL), berarti nggak ada kata slang dengan awalan tersebut. Namun kalau rutanya tembus sampai abjad awalan terakhir, saya jadikan letak titik ujung tersebut sebagai *start* awal pemanggilan fungsi DFS `displayAlphabetically`. Dengan begitu, kata yang keluar murni hanya yang berawalan sesuai *prefix* saja.

13. Menu 4 (View All), Menu 5 (Exit), dan Penutup

```

case 4: {
    if (dictionarySize == 0) {
        printf("\nThere is no slang word yet in the
dictionary.\n\n");
    } else {
        printf("\nList of all slang words in the
dictionary:\n");

        int currentNum = 1;
        displayAlphabetically(mainRoot, printBuffer, 0,
&currentNum);

        printf("\n");
    }
}

```

```

        }
        waitForEnter();
        break;
    }
    case 5: {
        printf("\nThank you... Have a nice day :)\n");
        break;
    }
    default: {
        printf("\nInvalid menu choice. Try again.\n");
        break;
    }
}
} while (userMenu != 5);

return 0;
}

```

Untuk menu keempat (*View All*), saya memasang pengecekan sederhana yang hanya melihat variabel global `dictionarySize`. Kalau variabel itu masih bernilai 0, program langsung ngasih peringatan kalau kamusnya belum ada isinya. Tapi jika lebih dari nol, pemanggilan fungsi `displayAlphabetically` bakal diluncurkan utuh dari *root* akar teratas, jadi seluruh cabang Trie bakal kecetak urut abjad. Terakhir, ada menu kelima buat *Exit*. Blok pengecekan utama `do-while` akan langsung berhenti saat nomor lima terpilih, program nge-print ucapan terima kasih, lalu berakhir bersih dengan perintah `return 0`.

Limitasi / Batasan Program

- Penggunaan Memori yang Boros (*Memory Inefficiency*):** Pada struktur data `SlangNode`, program menggunakan `array` `charPaths[MAX_CHARS]` berukuran 256 pointer untuk mengakomodasi seluruh kemungkinan karakter ASCII (huruf, angka, dan simbol). Hal ini sangat memakan memori komputer. Setiap kali ada satu karakter baru yang ditambahkan, program langsung memesan 256 jalur memori baru, meskipun pada praktiknya mungkin hanya 1 atau 2 jalur saja yang terpakai untuk kata berikutnya. Untuk kamus berskala kecil ini tidak masalah, tetapi jika database berisi jutaan kata, struktur Trie murni seperti ini akan sangat membebani RAM dibandingkan dengan penggunaan *Linked List* atau *Hash Map* pada cabangnya.
- Sensitivitas Huruf Kapital (*Case-Sensitivity*):** Karena algoritma Trie ini berpatokan ketat pada urutan angka kode ASCII, program akan menganggap huruf kapital dan huruf kecil sebagai dua karakter yang sepenuhnya berbeda. Jika *user* tidak sengaja menyimpan kata "MBG" dan "mbg", keduanya tidak akan menempa satu sama lain (sebagai *update*), melainkan tersimpan di dua cabang struktur Trie yang berbeda. Program ini belum menyertakan utilitas seperti `tolower()` untuk menstandarisasi semua *input* menjadi huruf kecil secara otomatis.
- Kerentanan pada Fungsi Pembersih Input (*flushInput*):** Fungsi buatan `flushInput()`

menggunakan perulangan `getchar()` yang secara logis berasumsi bahwa selalu ada sisa karakter (seperti `Enter/\n`) di dalam antrean memori masukan (*input buffer*). Jika pemanggilan fungsi ini tidak diletakkan dengan sangat hati-hati pada `main()`, dan terpanggil saat memori input sebenarnya sudah kosong, maka layar terminal akan terlihat menggantung (*hang*) karena sistem dipaksa menunggu ketikan baru dari pengguna yang tidak disadari oleh pengguna tersebut.

- **Keterbatasan Deteksi Spasi (Tanpa Regex):** Karena adanya larangan mutlak penggunaan *Regular Expressions* (Regex) pada persyaratan soal, deteksi format kata tanpa spasi sepenuhnya bergantung pada fungsi `isspace()`. Fungsi C++ ini sangat akurat mendeteksi spasi standar, tabulasi, atau *newline*, namun berpotensi gagal mengenali karakter-karakter spasi tersembunyi yang berasal dari *encoding* format Unicode/UTF-8 khusus (seperti *zero-width space*).
- **Risiko Stack Overflow pada Fungsi Rekursif (DFS):** Fungsi pencetakan kata secara alfabetis (`displayAlphabetically`) menggunakan metode pencarian menukik *Depth-First Search* (DFS) yang bersifat rekursif (fungsi yang memanggil dirinya sendiri masuk ke dalam node cabang). Untuk panjang maksimal kata saat ini (100 karakter), metode ini masih bekerja dengan sangat aman dan efisien. Namun, apabila konstanta maksimal karakter ini diubah menjadi sangat ekstrem (misal puluhan ribu karakter huruf), tumpukan pemanggilan rekursif ini dapat melebihi kapasitas memori sistem alokasi program dan memicu terjadinya *Stack Overflow Crash*.

Test Case Program

1. Insert 15 Slang (Screenshot di pisah karena tidak muat.)

```
23:05:09

=== Boogle Slang Dictionary ===
1. Release a new slang word
2. Search a slang word
3. View all slang words starting with a certain prefix word
4. View all slang words
5. Exit
Choose menu: 1
Input a new slang word [Must be more than 1 characters and contains no space]: slay
Input a new slang word description [Must be more than 1 word]: keren cakep

Successfully released new slang word.
Press enter to continue...

=== Boogle Slang Dictionary ===
1. Release a new slang word
2. Search a slang word
3. View all slang words starting with a certain prefix word
4. View all slang words
5. Exit
Choose menu: 1
Input a new slang word [Must be more than 1 characters and contains no space]: nocap
Input a new slang word description [Must be more than 1 word]: tidak bohong

Successfully released new slang word.
Press enter to continue...

=== Boogle Slang Dictionary ===
1. Release a new slang word
2. Search a slang word
3. View all slang words starting with a certain prefix word
4. View all slang words
5. Exit
Choose menu: 1
Input a new slang word [Must be more than 1 characters and contains no space]: mbg
Input a new slang word description [Must be more than 1 word]: mas bahlil ganteng

Successfully released new slang word.
Press enter to continue...

=== Boogle Slang Dictionary ===
1. Release a new slang word
2. Search a slang word
3. View all slang words starting with a certain prefix word
4. View all slang words
5. Exit
Choose menu: 1
Input a new slang word [Must be more than 1 characters and contains no space]: goat
Input a new slang word description [Must be more than 1 word]: greatest of all time

Successfully released new slang word.
Press enter to continue...
```

=== Boogle Slang Dictionary ===

1. Release a new slang word
2. Search a slang word
3. View all slang words starting with a certain prefix word
4. View all slang words
5. Exit

Choose menu: 1

Input a new slang word [Must be more than 1 characters and contains no space]: flex

Input a new slang word description [Must be more than 1 word]: pamer pamer

Successfully released new slang word.

Press enter to continue...

=== Boogle Slang Dictionary ===

1. Release a new slang word
2. Search a slang word
3. View all slang words starting with a certain prefix word
4. View all slang words
5. Exit

Choose menu: 1

Input a new slang word [Must be more than 1 characters and contains no space]: lowkey

Input a new slang word description [Must be more than 1 word]: diam diam

Successfully released new slang word.

Press enter to continue...

=== Boogle Slang Dictionary ===

1. Release a new slang word
2. Search a slang word
3. View all slang words starting with a certain prefix word
4. View all slang words
5. Exit

Choose menu: 1

Input a new slang word [Must be more than 1 characters and contains no space]: sus

Input a new slang word description [Must be more than 1 word]: mencurigakan banget coy

Successfully released new slang word.

Press enter to continue...

=== Boogle Slang Dictionary ===

1. Release a new slang word
2. Search a slang word
3. View all slang words starting with a certain prefix word
4. View all slang words
5. Exit

Choose menu: 1

Input a new slang word [Must be more than 1 characters and contains no space]: mid

Input a new slang word description [Must be more than 1 word]: b aja gitu

Successfully released new slang word.

Press enter to continue...

```
=== Boogle Slang Dictionary ===
1. Release a new slang word
2. Search a slang word
3. View all slang words starting with a certain prefix word
4. View all slang words
5. Exit
Choose menu: 1
Input a new slang word [Must be more than 1 characters and contains no space]: cringe
Input a new slang word description [Must be more than 1 word]: aneh geli

Successfully released new slang word.
Press enter to continue...
```

I

```
=== Boogle Slang Dictionary ===
1. Release a new slang word
2. Search a slang word
3. View all slang words starting with a certain prefix word
4. View all slang words
5. Exit
Choose menu: 1
Input a new slang word [Must be more than 1 characters and contains no space]: ghost
Input a new slang word description [Must be more than 1 word]: tiba tiba ngilang

Successfully released new slang word.
Press enter to continue...
```

```
=== Boogle Slang Dictionary ===
1. Release a new slang word
2. Search a slang word
3. View all slang words starting with a certain prefix word
4. View all slang words
5. Exit
Choose menu: 1
Input a new slang word [Must be more than 1 characters and contains no space]: aura
Input a new slang word description [Must be more than 1 word]: karisma seseorang

Successfully released new slang word.
Press enter to continue...
```

```
=== Boogle Slang Dictionary ===
1. Release a new slang word
2. Search a slang word
3. View all slang words starting with a certain prefix word
4. View all slang words
5. Exit
Choose menu: 1
Input a new slang word [Must be more than 1 characters and contains no space]: spillthetea
Input a new slang word description [Must be more than 1 word]: gossip gitu

Successfully released new slang word.
Press enter to continue...
```

2. Search 5 Kata

```
Input a slang word to be searched [Must be more than 1 characters and contains no space]: fomo
Slang word : fomo
Description : fear of missing out

Press enter to continue...

=== Boogle Slang Dictionary ===
1. Release a new slang word
2. Search a slang word
3. View all slang words starting with a certain prefix word
4. View all slang words
5. Exit
Choose menu: 2
Input a slang word to be searched [Must be more than 1 characters and contains no space]: mbg
Slang word : mbg
Description : mas bahlil ganteng

Press enter to continue...

=== Boogle Slang Dictionary ===
1. Release a new slang word
2. Search a slang word
3. View all slang words starting with a certain prefix word
4. View all slang words
5. Exit
Choose menu: 2
Input a slang word to be searched [Must be more than 1 characters and contains no space]: goat
Slang word : goat
Description : greatest of all time

Press enter to continue...

=== Boogle Slang Dictionary ===
1. Release a new slang word
2. Search a slang word
3. View all slang words starting with a certain prefix word
4. View all slang words
5. Exit
Choose menu: 2
Input a slang word to be searched [Must be more than 1 characters and contains no space]: spillthetea
Slang word : spillthetea
Description : gossip gitu

Press enter to continue...

=== Boogle Slang Dictionary ===
1. Release a new slang word
2. Search a slang word
3. View all slang words starting with a certain prefix word
4. View all slang words
5. Exit
Choose menu: 2
```

Input a slang word to be searched [Must be more than 1 characters and contains no space]: aura

Slang word : aura

Description : karisma seseorang

Press enter to continue...

=== Boogle Slang Dictionary ===

1. Release a new slang word
2. Search a slang word
3. View all slang words starting with a certain prefix word
4. View all slang words
5. Exit

Choose menu: █

2. View Prefix 5 Kata

```
=== Boogle Slang Dictionary ===
1. Release a new slang word
2. Search a slang word
3. View all slang words starting with a certain prefix word
4. View all slang words
5. Exit
Choose menu: 3
Input a prefix to be searched: s

Words starts with "s":
1. slay
2. spillthetea
3. sus

Press enter to continue...

=== Boogle Slang Dictionary ===
1. Release a new slang word
2. Search a slang word
3. View all slang words starting with a certain prefix word
4. View all slang words
5. Exit
Choose menu: 3
Input a prefix to be searched: m

Words starts with "m":
1. mbg
2. mid

Press enter to continue...

=== Boogle Slang Dictionary ===
1. Release a new slang word
2. Search a slang word
3. View all slang words starting with a certain prefix word
4. View all slang words
5. Exit
Choose menu: 3
Input a prefix to be searched: f

Words starts with "f":
1. flex
2. fomo

Press enter to continue...
```

```
=== Boogle Slang Dictionary ===
```

1. Release a new slang word
2. Search a slang word
3. View all slang words starting with a certain prefix word
4. View all slang words
5. Exit

Choose menu: 3

Input a prefix to be searched: g

Words starts with "g":

1. ghost
2. goat

Press enter to continue...

```
=== Boogle Slang Dictionary ===
```

1. Release a new slang word
2. Search a slang word
3. View all slang words starting with a certain prefix word
4. View all slang words
5. Exit

Choose menu: 3

Input a prefix to be searched: a

Words starts with "a":

1. aura

Press enter to continue...

■

3. View All

Press enter to continue...

=== Boogle Slang Dictionary ===

1. Release a new slang word
2. Search a slang word
3. View all slang words starting with a certain prefix word
4. View all slang words
5. Exit

Choose menu: 4

List of all slang words in the dictionary:

1. aura
2. cringe
3. flex
4. fomo
5. ghost
6. goat
7. highkey
8. lowkey
9. mbg
10. mid
11. nocap
12. omg
13. slay
14. spillthetea
15. sus

Press enter to continue...

